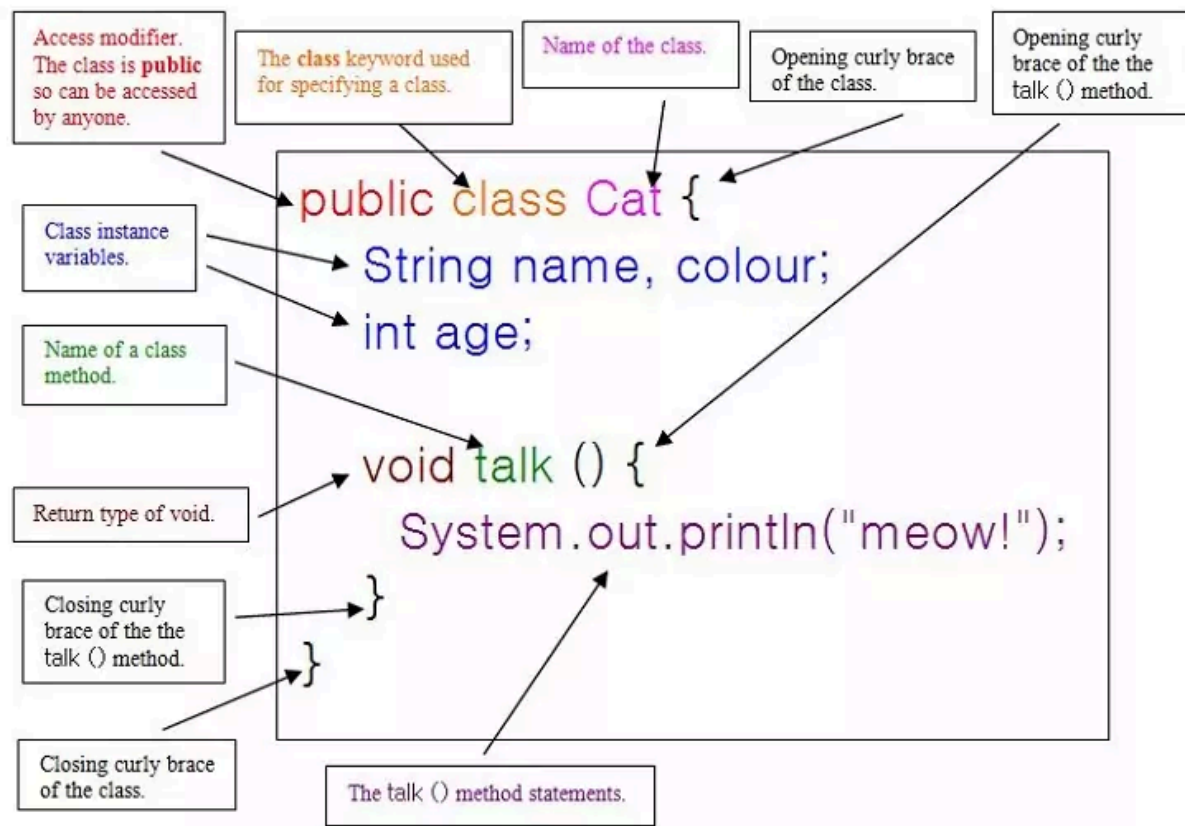


In Java, all code must reside inside a class. By convention, the name of the main class should match the name of the file that holds the program. You should also make sure that the capitalization of the filename matches the class name. The reason for this is that Java is case-sensitive.



1 The rule depends on whether the class is public or not public

✓ Case 1: If the class is declared **public**

Then the **filename MUST match the class name exactly**, including **capitalization**.

```
// File: Example.java
public class Example {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

✓ Works fine.

Now, if you rename the file:

File: example.java

and try to compile:

```
javac example.java
```

❌ You'll get an **error** like:

error: class Example is public, should be declared in a file named Example.java

So when a class is **public**, filename and class name must **match exactly (case-sensitive)**.

✅ Case 2: If the class is NOT declared **public**

Then the filename can be **anything** — Java doesn't enforce the name match.

```
// File: TestFile.java
class Example {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Compile:

```
javac TestFile.java
java Example
```

✅ Works perfectly fine.

The compiler doesn't care about the filename in this case, only about class names and the **main** method.

🧩 2 Why the convention exists

Even though Java allows flexibility for non-public classes, **developers always follow the convention**:

- Public class name = filename

- Capitalization matches exactly

Why:

- It avoids confusion in large projects
- IDEs (like IntelliJ, Eclipse, VS Code) rely on this convention
- Easier navigation and file management
- Mandatory if you ever make the class `public` later

✓ In short:

Case	Class Modifier	Must Match File Name?	Example
1	<code>public class Example</code>	✓ Yes (case-sensitive)	<code>Example.java</code>
2	<code>class Example</code>	✗ No (but recommended)	<code>TestFile.java</code>

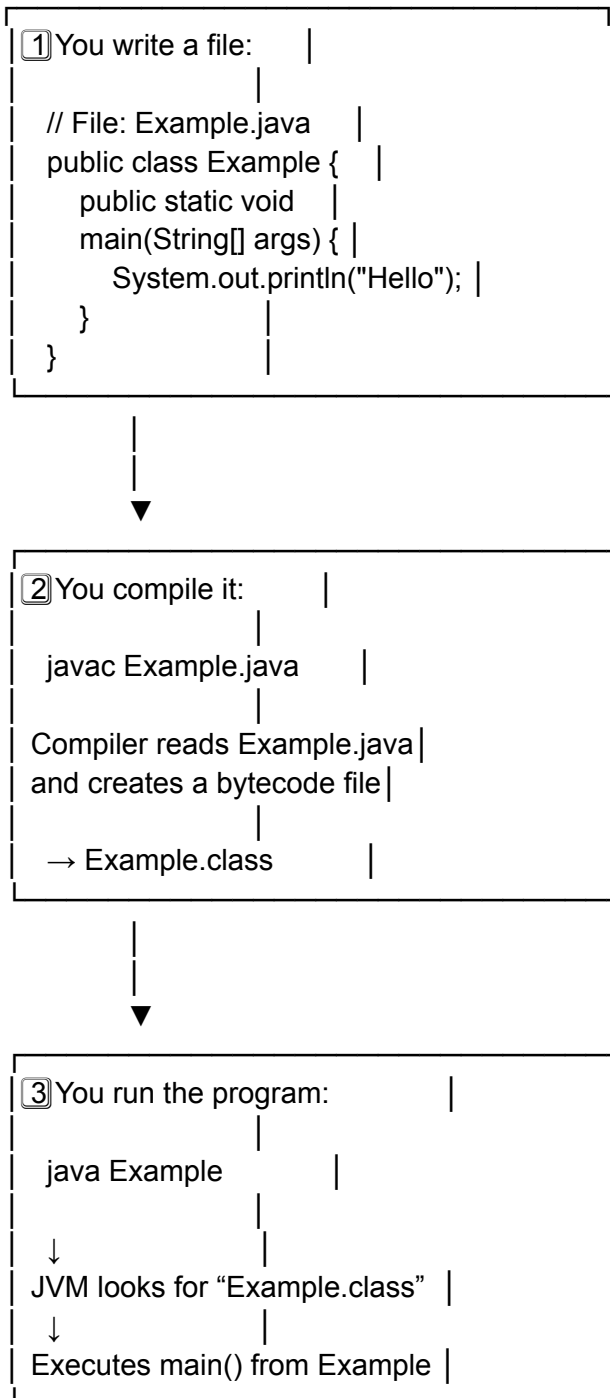
The `javac` compiler creates a file called `Example.class` that contains the bytecode version of the program. Java bytecode is the intermediate representation of your program that contains instructions the Java Virtual Machine will execute. Thus, the output of `javac` is not code that can be directly executed. To actually run the program, you must use the Java application launcher called `java`.

When Java source code is compiled, each individual class is put into its own output file named after the class and using the `.class` extension. This is why it is a good idea to give your Java source files the same name as the class they contain—the name of the source file will match the name of the `.class` file. When you execute `java` as just shown, you are actually specifying the name of the class that you want to execute. It will automatically search for a file by that name that has the `.class` extension. If it finds the file, it will execute the code contained in the specified class.

Perfect question — you’ve reached the point where most learners get confused between **source file names**, **class names**, and **.class files**.

Let’s visualize the **flow** of what actually happens from writing → compiling → running 📌

🧩 Step-by-Step Visual Flow



Key Points:

Stage	Input	Output	Naming Rule
Source	<code>Example.java</code>	—	Must match public class name
Compile	<code>javac</code> <code>Example.java</code>	<code>Example.class</code>	Compiler auto-generates file with class name
Run	<code>java Example</code>	Executes bytecode	JVM searches for <code>Example.class</code>

What if you name file differently?

Example:

```
// File: TestFile.java
public class Example {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Try:

```
javac TestFile.java
```

 Error:

class Example is public, should be declared in a file named Example.java

So the **JVM doesn't care about the source filename**,
but the **compiler enforces filename = public class name** because it must know where to store the `.class` file.

Quick Mental Model:

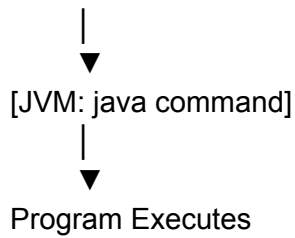
Source Code (.java)



[Compiler: javac]

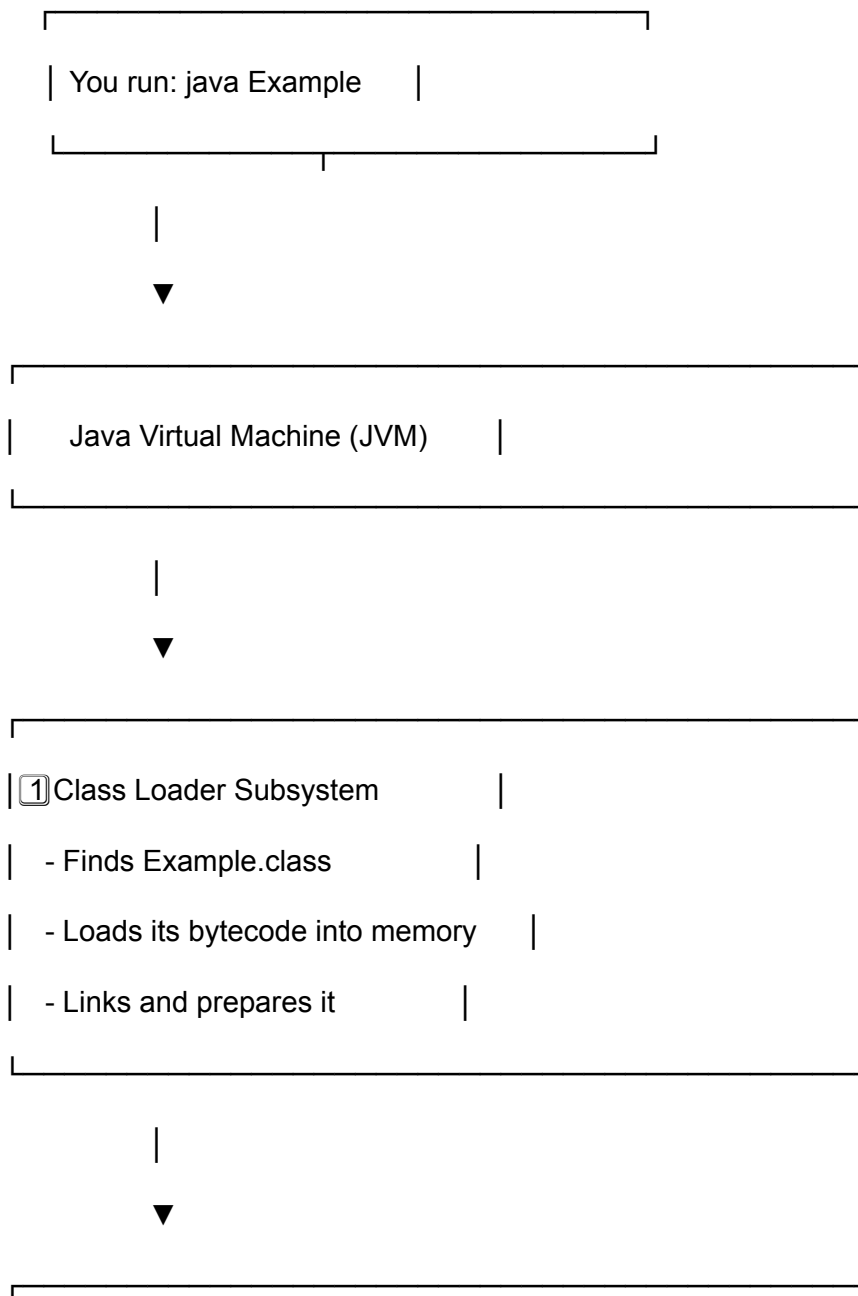


Bytecode (.class)



Let's continue and visualize **what happens *inside the JVM*** after you run `java Example`

Inside the JVM — How Your Code Actually Runs



| ② Bytecode Verifier |

- | - Checks for illegal code |
- | (e.g., stack overflow, wrong types) |
- | - Ensures bytecode is safe |

|



| ③ Just-In-Time (JIT) Compiler |

- | - Converts bytecode → native machine code |
- | - Optimizes frequently used methods |
- | - Speeds up execution |

|



| ④ Execution Engine |

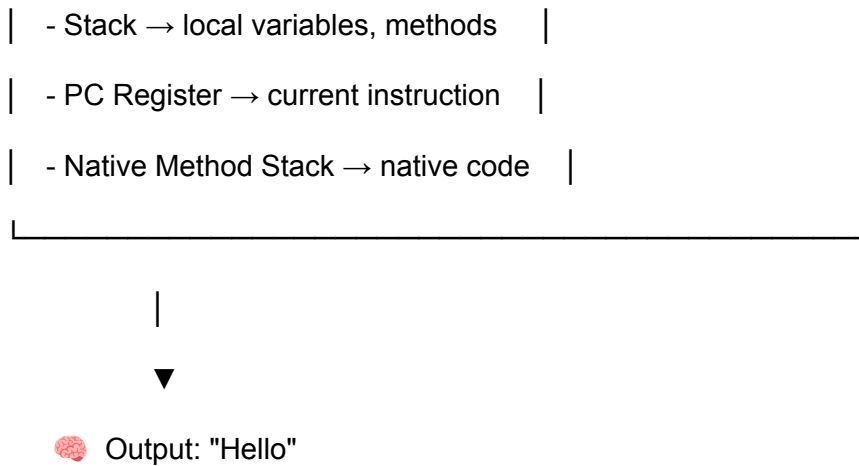
- | - Runs native machine code on CPU |
- | - Uses OS and memory resources |

|



| ⑤ Runtime Data Areas |

- | - Method Area → class info, static vars |
- | - Heap → objects live here |



Putting it all together visually

Source Code (.java)



Compiler (javac)



Bytecode (.class)



JVM

└─ Class Loader

└─ Bytecode Verifier

└─ JIT Compiler

└─ Execution Engine

└─ Runtime Data Areas





 Program Executes!



Summary of the Flow

Stage	Role	Output
javac	Compiles <code>.java</code> → <code>.class</code>	Bytecode
Class Loader	Loads <code>.class</code> into JVM	In-memory bytecode
Verifier	Checks bytecode safety	Verified bytecode
JIT Compiler	Converts to machine code	Optimized native code
Execution Engine	Runs it on CPU	Actual output
Runtime Areas	Store program data	Memory management

`main()` must be declared as `public`, since it must be called by code outside of its class when the program is started. The keyword `static` allows `main()` to be called without having to instantiate a particular instance of the class. This is necessary since `main()` is called by the Java Virtual Machine before any objects are made.



First, recall what happens when you run:

java Example

The **JVM** starts your program by looking for:

```
public static void main(String[] args)
```

It does this **before creating any objects** or running any other code.

Let's break the sentence down piece by piece

🧩 1 “**main()** must be declared as **public**”

✅ Why?

Because when the JVM starts your program, it's *outside* your class.
It needs to **access** your method from *outside*.

- Think of **public** as “door open to everyone.”
- If it were **private** or **default**, the JVM couldn't find or access it.

Example:

```
class Example {  
    private static void main(String[] args) { } // ❌ JVM can't access this  
}
```

Output:

Error: Main method not found in class Example

So → **public = visible to JVM** ✅

🧩 2 “**static** allows **main()** to be called without instantiating the class”

✅ Why?

Normally, in Java, you must create an object to call a method:

```
class Dog {  
    void bark() { System.out.println("woof"); }  
}
```

```
Dog d = new Dog(); // create an object  
d.bark();           // call method
```

But the JVM **has not yet created any objects** when it starts your program.
It just knows the **class name** (**Example**).

So **main()** must be **static**, because it can then be called using only the class name:

```
Example.main(); // JVM does this internally
```

No object creation required.

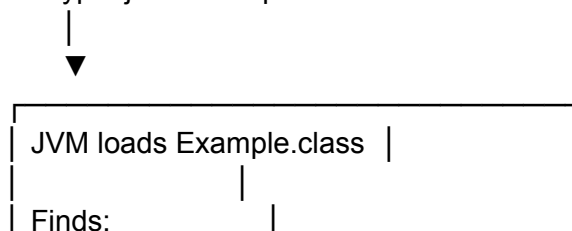
🧩**3** “This is necessary since **main()** is called by the JVM before any objects are made”

✅ In simple words:

- When you run **java Example**, the **JVM**:
 1. Loads the **Example.class** file.
 2. Looks for a method called **public static void main(String[] args)**.
 3. Calls it **without creating an Example object**.
 - Only **inside** that **main()** can you create objects and call other methods.
-

🎯 Putting it all together visually

You type: **java Example**



```

| public static void main(String[] args) |
|                                     |
| Calls: Example.main() | ← no object created
|                                     |
| Inside main(): you can create objects |
| e.g. Example e = new Example(); |
|_____|

```

Summary Table

Keyword	Why it's needed in <code>main()</code>	Analogy
<code>public</code>	JVM (outside class) must access it	"The door is open for JVM" 🚪
<code>static</code>	JVM can call it without making an object	"Call directly using class name" 📞
<code>void</code>	It doesn't return anything	"Just does its job and exits" 🚶

So in essence:

 `public` lets the JVM **find** your main method,
`static` lets it **call** it *without creating an object*,
 and `void` means it **doesn't return** anything back to JVM.

Java compiler will compile classes that do not contain a `main()` method. But java has no way to run these classes. So, if you had typed `Main` instead of `main`, the compiler would still compile your program. However, java would report an error because it would be unable to find the `main()` method.

Step 1 — Understand that Compilation ≠ Execution

When you do:

```
javac Example.java
```

→ The **compiler** (`javac`) only checks for:

- syntax correctness,
- data types,
- missing semicolons,
- invalid keywords, etc.

✅ It **does not care** if you have a `main()` method or not.

Its only job is:

👉 “Turn your `.java` file into `.class` (bytecode) file.”

🧩 Step 2 — Example: class without `main()`

// File: Helper.java

```
public class Helper {  
    void greet() {  
        System.out.println("Hello");  
    }  
}
```

Now run:

```
javac Helper.java
```

✅ Output: no errors, and it produces → `Helper.class`

But if you try to **run** it:

```
java Helper
```

❌ Output:

Error: Main method not found in class Helper, please define the main method as:

```
public static void main(String[] args)
```

So What Happened?

Stage	What Tool Does	What It Checks	Does It Need <code>main()</code> ?
Compilation	<code>javac</code>	Syntax, type safety	 No
Execution	<code>java</code> (JVM)	Looks for <code>public static void main(String[] args)</code>	 Yes

Step 3 — Case Sensitivity Example

If you accidentally write:

```
public static void Main(String[] args) { } // notice M is uppercase
```

Compile:

```
javac Example.java
```

 Compiles successfully → creates `Example.class`

Run:

```
java Example
```

 Error:

Error: Main method not found in class Example

Why?

Because:

- Java compiler doesn't check *if* `main()` exists — it just checks that your syntax is correct.

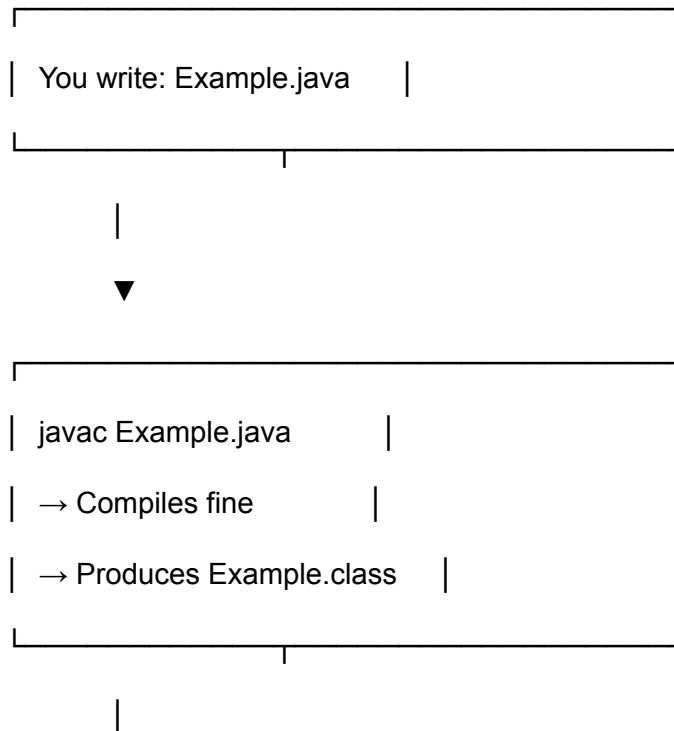
But **the JVM**, during execution, specifically searches for a method named **exactly**:

```
public static void main(String[] args)
```

- — lowercase **m**, exact parameter type, exact modifiers.

If it doesn't find that signature → it can't start the program.

Step 4 — Visual Summary





java Example	
→ JVM starts	
→ Looks for public static	
void main(String[])	
❌ If not found → Error	

✅ So, simplified:

The **compiler (javac)** only checks if your grammar is right.

The **JVM (java)** needs a `main()` to *know where to begin*.

If it's missing or misnamed (like `Main`), JVM says:

"I compiled your movie, but I don't know which scene to start with!"

The main method signature

```
public static void main(String[] args)
```

Now break it apart:

- `public` → JVM must access it
- `static` → JVM can call it without creating an object
- `void` → returns nothing
- `main` → name of the method JVM looks for

- `String[] args` → parameter (input) that receives **command-line arguments**
-

So what is `String[] args`?

It means:

"`args` is an array of Strings."

What is an Array?

An array is just a **collection of values** stored together.

Example:

```
int[] numbers = {1, 2, 3};
```

```
String[] fruits = {"apple", "banana", "mango"};
```

So:

```
String[] args
```

is an array that can store **multiple strings** — each string comes from **the command line** when you run your program.

Visualizing it

Imagine you run this in your terminal:

```
java Example hello world java
```

Then:

```
args[0] = "hello"
```

```
args[1] = "world"
```

```
args[2] = "java"
```

Example program

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Number of arguments: " + args.length);  
  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("args[" + i + "] = " + args[i]);  
        }  
    }  
}
```

Run it like this:

```
java Example Sora Samuela Sara
```

Output:

```
Number of arguments: 3
```

```
args[0] = Sora
```

```
args[1] = Samuela
```

```
args[2] = Sara
```

Key idea:

Term	Meaning
<code>String</code>	Type → sequence of characters (like "Hello")
<code>String[]</code>	Array of strings
<code>args</code>	Variable name for that array
Command-line arguments	Words you type <i>after</i> class name when running the program

Visual Summary

Command line input:

```
java Example red blue green
```

JVM passes this to main():

```
args[0] = "red"
```

```
args[1] = "blue"
```

```
args[2] = "green"
```

main(String[] args) receives:

```
["red", "blue", "green"]
```

So when your textbook says:

“args receives any command-line arguments present when the program is executed.”

It simply means:

Whatever words you type after your program name in the terminal are automatically stored inside the `args` array for your program to use.
